Action Runtime

Purpose

Action Runtime is responsible for every interaction between Shio and the outside world.

Its responsibility is not limited to executing Tool Calls.

It is the only Runtime allowed to produce observable side effects outside of the digital character.

Everything that changes the external world must pass through Action Runtime.

This architectural boundary guarantees safety, consistency and replaceability.

Why Action Runtime Exists

Language models should never directly interact with external systems.

They should never:

- access APIs;
- modify files;
- execute commands;
- interact with streaming platforms;
- change application settings.

Instead, the language model expresses an intention.

Action Runtime decides whether that intention may become reality.

Responsibility

Action Runtime owns:

- Tool Registry;
- Tool Discovery;
- Tool Validation;
- Permission Guard;
- Tool Execution;
- Action Logging;
- External Adapters;
- Execution Results.

Its responsibility ends when the requested action has either:

- completed successfully;
- failed;
- been rejected.

What happens afterwards belongs to Experience Layer.

Architectural Boundary

The Runtime is divided into two worlds.

Internal World

Everything that exists inside the character.

Examples:

- personality;
- emotions;
- memory;
- conversation;
- animation;
- speech.

These systems cannot directly modify external reality.

External World

Everything outside the Runtime.

Examples:

- operating system;
- files;
- YouTube;
- Twitch;
- Discord;
- OBS;
- HTTP APIs;
- local services;
- smart home devices.

Only Action Runtime communicates across this boundary.

The Action Boundary

```text id="act01" Character

---

Speech Runtime

Experience Layer

Avatar Runtime

Memory Layer

---

    Action Runtime

---

External World
```

```
Action Runtime acts as the controlled gateway between the character and
reality.

---

# Tool Registry

Every executable capability is represented as a Tool.

Examples include:

- BanViewer
- ChangeScene
- PlaySound
- ToggleOverlay
- OpenApplication
- Future Integrations

The Runtime owns the registry.

The LLM never owns available capabilities.

---

# Permission Guard
```

````
Every action passes through Permission Guard before execution.

Permission Guard determines whether an action is allowed.

Examples include:

- user permissions;
- safety policies;
- application state;
- platform limitations.

Permission decisions are deterministic.

The LLM does not participate.

---

# Action Lifecycle

Every action follows the same lifecycle.

```text id="act02"
Requested

↓

Validated

↓

Authorized

↓

Executed

↓

Completed

↓

Experience Generated
````

This lifecycle remains identical regardless of the action itself.

````

Action Results

Every action produces a result.

Typical outcomes include:

- Success
- Failure
- PermissionDenied
- Timeout
- Cancelled

These results become inputs to Experience Layer.

Action Runtime itself never decides how the character emotionally reacts.

External Adapters

Action Runtime never communicates directly with concrete platforms.

Instead it delegates execution to Adapters.

Examples:

```text id="act03" BanViewer

↓

YouTube Adapter

or

Twitch Adapter

or

Discord Adapter
```

```
The action remains identical.

Only the Adapter changes.

This allows new platforms to be added without modifying Action Runtime.

---
```

```
# Platform Independence

The architecture deliberately separates actions from platforms.

For example:

The action:

```text id="act04"
BanViewer
```

should remain identical regardless of whether the target platform is:

- YouTube;
- Twitch;
- Discord;
- future services.

Platform-specific implementation belongs entirely to the corresponding Adapter.

Character Independence

The character never knows how actions are implemented.

Shio may intend to:

> "Ban this spammer."

The Runtime determines:

- which Tool executes;
- which Adapter is selected;
- how the API call is performed;
- how retries are handled.

The character remains completely unaware of implementation details.

Relationship With Experience Layer

Action Runtime executes.

Experience Layer interprets.

This distinction is fundamental.

For example:
```

```text id="act05" Tool Completed

↓

Experience Layer

↓

Smirk

↓

Short Joke

↓

Return To Neutral ```

The same action could produce entirely different reactions depending on personality, context and memory.

---

# Future Evolution

Action Runtime is expected to become the execution platform for every future capability of Shio.

Potential future actions include:

- moderation;
- application control;
- automation;
- scheduling;
- content creation;
- external integrations;
- smart device control.

Regardless of future complexity, one architectural rule remains unchanged:

**No external side effect may bypass Action Runtime.**
```